Localization with Livox MID-360 3D LiDAR

By: >*terminal_gremlin (c-lydia^^)*

Source

- Authors/lecturer:
  - Jiarong Lin, Fu Zhang (Loam Livox, FAST_LIO)
  - Tixiao Shan, Grendan Englot, Drew Meyers, Wei Wang, Carlo Ratti, Daniela Rus (LIO-SAM)
  - Tom Moore, David Stouch (robot_localization)
  - Lvox-SDK (livox_relocalization)
- Year:
  - robot_localization: 2014
  - Loam Livox: 2019
  - LIO-SAM, FAST_LIO: 2020
  - livox_relocalization: 2019-2021
- URL/DOI/PDF:
  - Loam livox: A fast, high-precision LiDAR odometry and mapping package for LiDARs of small FoV, Jiarong Lin and Fu Zhang
  - LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping, Tixiao Shan, Brendan Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus
  - GitHub - hku-mars/FAST_LIO: A computationally efficient and robust LiDAR-inertial odometry (LIO) package · GitHub
  - GitHub - Livox-SDK/livox_relocalization: A relocalization package for Livox LiDARs. · GitHub
  - robot_localization wiki — robot_localization 2.7.7 documentation

Question

How do I localize the robot with a 3D lidar?

Core idea

3D LiDAR localization estimates a robot's 6 Degree of Freedom (DoF) pose ($X, Y, Z, roll, pitch, yaw$) by continuously aligning incoming 3D point cloud scans with local or global maps. To achieve accurate, real-time tracking, the system must compensate for sensor motion blur (in-frame skewing) using synchronized IMU measurements, extract structural geometric features (edges, planes) or match raw points directly against dynamic

spatial search structures, and suppress long-term odometric drift using factor graphs or non-linear kalman filtering (`robot_localization, FAST-LIO`).

Method and assumptions

Key Algorithmic Steps

1. **Front-End Feature Extraction and Point Selection:**
   - Extract edge and planar feature points based on loal surface roughness or reflectivity transitions across materials
   - Filter out unreliable points (e.g., FOV fringe regions, high or low intensity noise, extreme beam incident angles, or occluded background points)
   - Modern direct methods (FAST-LIO2) can operate directly on raw point clouds without feature extraction
2. **In-Frame Motion Compensation (De-skewing):**
   - Correct motion blur caused by continuous laser scanning using high-rate IMU pre-integration motion priors, piecewise frame splitting across parallel CPU threads, or linear interpolation
3. **Scan Matching and Pose Optimization:**
   - **Iterative Point-to-Edge and Point-to-Plane ICP:** formulate distance residuals between extracted feature points and map keyframes or sub-keyframes using $KD-Tree$ or $ikd-Tree$
   - **Tightly-Coupled Factor Graphs:** insert IMU pre-integration factors, LiDAR odometry factors, loop closure factors, and GPS factors into a global factor graph optimized via incremental smoothing and mapping (iSAM2)
   - **Iterated Extended Kalman Filter (IEKF):** update state vectors recursively on manifold spaces (FAST-LIO)
   - **Global or Relocalization Matching:** load pre-built `.pcd` maps (e.g., `all_points.pcd`, `corner.pcd`, `surf.pcd`) to align incoming scans for initial global positioning
4. **Multi-Sensor Fusion Integration:**
   - Fuse state estimates across multiple odometry sources, IMUs, and GPS streams using a unified 15-dimensional EKF or UKF filter node (`robot_localization`)

Assumptions

- The environment contains sufficient geometric structure (surfaces or lines) to constrain scan matching
- LiDAR and IMU sensors are rigidly mounted, calibrated, and hardware or software synchronized

- Accelerations and angular velocities remain reasonably smooth within tiny integration steps

Equations and notation

Defines Symbols and Units

- $\mathbf{P} = [x, y, z]^T \in \mathbb{R}^3$: 3D point coordinates in the LiDAR frame (meters)
- $D(\mathbf{P}) = \sqrt{x^2 + y^2 + z^2}$: depth or distance to sensor origin (meters)
- $\phi(\mathbf{P}) = \arctan(\frac{y^2+z^2}{x})$: deflection angle relative to the sensor boresight x-axis (rad/degrees)
- $\theta(\mathbf{P})$: bean incident angle between laser ray and the local target plane
- $\mathbf{R} \in SO(3)$: rotation matrix
- $\mathbf{p} \in \mathbb{R}^3$: position vector in world frame
- $\mathbf{W}$: world frame
- $\mathbf{v} \in \mathbb{R}^3$: linear vcelocity
- $\mathbf{b}$: IMU bias
- $\mathbf{x} = [\mathbf{R}, \mathbf{p}, \mathbf{v}, \mathbf{b}]^T$: robot state vector
- $\mathbf{P}_w = \mathbf{R}_k \mathbf{P}_l + \mathbf{T}_k$: projected point in the global maop frame

Core Fomulations

Edge-to-Edge Point Distance Residual (r_{e2e})

For an edge feature point $\mathbf{P}_w$ transformed into the global map, locate the 5 nearest map edge points forming a line between $\mathbf{P}_1$ and $\mathbf{P}_5$, we have

$$r_{e2e} = \frac{||(\mathbf{P}_w - \mathbf{P}_1) \times (\mathbf{P}_w - \mathbf{P}_5)||}{||\mathbf{P}_1 - \mathbf{P}_5||} \quad [m]$$

Plane-to-Plane Point Distance Residual (r_{p2p})

For a planar feature point $\mathbf{P}_w$, locate 5 nearest planar map points ($\mathbf{P}_1, \mathbf{P}_3, \mathbf{P}_5$) defining a planar patch as

$$r_{p2p} = \frac{(\mathbf{P}_w - \mathbf{P}_1)^T [(\mathbf{P}_3 - \mathbf{P}_1) \times (\mathbf{P}_5 - \mathbf{P}_1)]}{|(\mathbf{P}_3 - \mathbf{P}_1) \times (\mathbf{P}_5 - \mathbf{P}_1)|} \quad [m]$$

IMU Pre-integration Relative Motion Delta

Pre-integrated rotation ($\Delta\mathbf{R}_{i,j}$), velocity ($\Delta\mathbf{v}_{i,j}$), and position ($\Delta\mathbf{p}_{i,j}$) deltas between timestamps i and j is

$$\Delta \mathbf{R}_{i,j} = \mathbf{R}_i^T \mathbf{R}_j$$

$$\Delta \mathbf{v}_{i,j} = \mathbf{R}_i^T (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g}\Delta t_{i,j}^2)$$

$$\Delta \mathbf{p}_{i,j} = \mathbf{R}_i^T (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{i,j} - \frac{1}{2}\mathbf{g}\Delta t_{i,j}^2)$$

15-Dimensional State Vector (`robot_localization`)

State vector tracked by EKF or UKF state estimation nodes is:

$$\begin{aligned}\mathbf{x}_{15D} = \Big(&\mathbf{X}, \mathbf{Y}, \mathbf{Z}, \mathbf{roll}, \mathbf{pitch}, \mathbf{yaw},\\ &\dot{\mathbf{X}}, \dot{\mathbf{Y}}, \dot{\mathbf{Z}}, \dot{\mathbf{roll}}, \dot{\mathbf{pitch}}, \dot{\mathbf{yaw}},\\ &\ddot{\mathbf{X}}, \ddot{\mathbf{Y}}, \ddot{\mathbf{Z}}\Big)\end{aligned}$$

Evidence and limitations

Author Claims and Benchmark Evidence

- **Loam Livox:**
  - Achieves real-time odometry and mapping at $20Hz$ on CPUs
  - Multi-threaded sub-frame parallel matching runs $2-3\times$ faster than standard LOAM baseline ($17.24ms$ vs $109.00ms$ per frame)
  - Evaluated against Motion Capture (MoCap) systems with an average Euler angle orientation error of $\approx 1.1^o$, and translational drift against GPS of 0.41 % - 0.65 %
- **LIO-SAM:**
  - Processes scans up to $13\times$ faster than real-time on Intel i7 CPUs
  - Reduces end-to-end translational error from $> 120m$ (standmay fail to extract sufficient geometric features in long corridors, open fields, or featureless indoor spacesard LOAM) down to $0.12m$ on campus loops and $0.04m$ in park environments when loop closure and GPS factors are active
- **FAST-LIO:**
  - Delivers state estimation rates exceeding $100Hz$ using dynamic ikd-Trees
  - Supports spinning LiDARs (Velodyne, Ouster) and solid-state LiDARs (Livox Avia, Horizon, MID-70) on low-power ARM platforms (e.g., Raspberry Pi 4B, Jetson TX2)

Limitations and Edge Cases

- **Small FoV and Feature Degeneration:** solid-state sensors (e.g., Livox Mid-40 with 38.4^o FoV) yield fewer structural features per scan, making registration susceptible to moving obstacles or featureless corridors

- **Uncorrected Drift:** pure LiDAR-inertal odometry accumulates drift over long durations if loop closures or absolute constraints (GPS, compass, alimeter) are absent
- **Relocalization Initialization:** relocalization algorithms (such as `livox_relocalization`) require initial pose estimates to be close to the original map trajectory, particularly in orientation, to converge correctly
- **Initialization Failures:** loosely-coupled visual-inertial or optimization-based pipelines (e.g., LIOM) fail to initialize under aggressive rotational maneuvers or jerky motions

Connections

Related Concepts and Frameworks

- **LOAM pr LeGO-LOAM:** foundational feature-based LiDAR odometry baselines using point-to-edge and point-to-plane optimization
- **iSAM2 or Fractor Graphs:** bayes tree incremental optimization for efficient multi-sensor fusion
- **ikd-Tree:** dunamic spatial indexing structure supporting fast k-NN searches and concurrent point insertions or deletions
- **`robot_localization`:** standard ROS system package fusing heterogeneous sensor data sreams (IMU, wheel odometry, GPS via `navsat_transform_node`)

Follow-up

- What is ICP?
- What is KD-Tree and ikd-Tree?
- What is factor graph optimization?